Universidad Cooperativa de Colombia

ScoutLab
Sistema de apoyo al scouting deportivo con Random Forest

Evaluación final de Minería de Datos
Metodología CRISP-DM

Dody Cortez Duarte
Miguel Zarza

Docente
ROYER ELIECER MEJIA VILLALBA

31 de octubre de 2026

Repositorio del proyecto
https://github.com/DodyCe/scoutlab

RESUMEN

Se desarrolló un sistema académico que estima si un jugador adulto pertenece al segmento de mayor valor de mercado. El flujo integra MySQL, preparación de datos, comparación de árboles, optimización de un bosque de 100 árboles, persistencia y consumo desde una página web. Se reservaron datos de prueba y se exportó exactamente el modelo evaluado. El bosque optimizado alcanzó F1 de 0,5001 y AUC de 0,6921. El resultado permite orientar una revisión inicial, aunque su precisión de 36,86 % exige verificación humana. La aplicación valida la edad mínima de 18 años y funciona sin consultar MySQL durante la inferencia.

Palabras clave: CRISP-DM, scouting, Random Forest, FastAPI, clasificación.

I INTRODUCCIÓN Y OBJETIVOS

El proyecto sigue CRISP-DM al definir la necesidad de scouting, comprender y preparar los datos, comparar modelos, evaluar sus errores y desplegar el bosque en el aplicativo. Se busca priorizar perfiles mediante edad, estatura y posición. La etiqueta élite identifica un segmento de valor comercial (Mejia Villalba, 2026a).

El objetivo general es construir y comprobar el flujo desde los datos relacionales hasta una consulta web. Los objetivos específicos son preparar datos consistentes, comparar modelos con una prueba reservada y entregar predicciones a través de una API, conservando el tratamiento aprendido durante el entrenamiento.

II MARCO TEÓRICO

En el aprendizaje supervisado, el modelo usa ejemplos con una etiqueta conocida. Aquí emplea edad, estatura y posición para estimar si un jugador pertenece al grupo de mayor valor de mercado.

Un árbol muy profundo puede memorizar detalles de los datos. La poda limita su crecimiento y Random Forest combina varios árboles para reducir la dependencia de uno solo (Breiman, 2001). Los jugadores reservados permiten evaluar si el modelo generaliza.

Glosario

A Preparación de los datos

Imputación por mediana agrupada. Completa las estaturas faltantes con la mediana de la posición correspondiente y usa la mediana global como respaldo. La mediana ofrece menor sensibilidad a valores extremos que la media.

Target por percentil. El percentil 75, o Q3, delimita la etiqueta binaria: un valor de mercado igual o superior al límite se clasifica como élite. Los empates pueden producir algo más de un 25 % de positivos.

One-hot encoding. Representa cada posición con indicadores binarios. Su ajuste y las categorías aprendidas se conservan para que las columnas de un jugador nuevo sean las mismas que vio el modelo.

Muestreo estratificado. Divide Train y Test manteniendo aproximadamente la proporción de las clases, con el redondeo propio del número de registros. En este proyecto se usa una partición 80/20.

B Diagnóstico y optimización

Sobreajuste. Es la dependencia excesiva de detalles del entrenamiento que no se mantienen en datos nuevos. La diferencia entre el rendimiento de Train y Test ayuda a diagnosticarlo.

Poda previa. Limita el árbol con max_depth (profundidad máxima) y min_samples_leaf (mínimo de muestras por hoja). Reduce complejidad; una brecha pequeña por sí sola no garantiza detectar correctamente a la clase minoritaria.

Paradoja de la exactitud. Si la mayoría de los jugadores son estándar, predecir siempre esa clase puede producir accuracy alto y recall de élite nulo. Se necesita comparar con ese modelo trivial y revisar errores por clase.

Pesos balanceados. class_weight="balanced" asigna mayor peso por observación a las clases menos frecuentes. Cambia la contribución de los ejemplos al entrenamiento; no crea nuevos jugadores.

Random Forest y bagging. Un bosque combina árboles entrenados con muestras bootstrap y subconjuntos aleatorios de variables. Esa diversidad permite reducir la dependencia de un solo árbol, aunque no garantiza mejorar todas las métricas (Breiman, 2001). En scikit-learn, la clasificación se obtiene a partir del promedio de probabilidades de los árboles (scikit-learn developers, s.f.-d).

GridSearchCV. Compara combinaciones de hiperparámetros mediante validación cruzada y selecciona la que maximiza la métrica definida. Aquí se emplean cinco pliegues estratificados y F1 para la clase élite.

C Evaluación avanzada

ROC y AUC. La curva relaciona recall o tasa de verdaderos positivos con la tasa de falsos positivos al variar el umbral. AUC resume la discriminación: 0,5 es la referencia aleatoria y 1 es separación perfecta. Puede ser inferior a 0,5 si el ordenamiento es peor que esa referencia (scikit-learn developers, s.f.-b).

Importancia Gini. Resume la reducción de impureza atribuida a cada variable en el bosque. Describe el uso de las variables por el modelo, no sus efectos causales, y puede favorecer atributos con más posibilidades de división (scikit-learn developers, s.f.-d).

D Persistencia y consumo

Serialización con Joblib. Guarda objetos de Python para volver a cargarlos. Se guardan el bosque y las reglas de preparación, además del percentil y los metadatos, para mantener la consistencia entre entrenamiento e inferencia (Mejia Villalba, 2026b; scikit-learn developers, s.f.-c).

API REST y cliente-servidor. La página envía datos en JSON mediante HTTP. FastAPI valida la entrada con Pydantic, ejecuta el modelo y devuelve clase y probabilidades. POST /predict es el punto de acceso usado por el formulario (FastAPI, s.f.).

III DATOS Y MÉTODO

La ejecución del 2026-10-31 consultó 41.438 jugadores de 18 años o más. Se reservaron 8.288 para Test y se usaron 33.150 para Train. Hubo 2.167 estaturas faltantes. La etiqueta se fijó en Q75 = EUR 700.000; quedaron 30.382 estándar y 11.056 élites.

Se mantuvo la definición global del percentil indicada por la guía. Las medianas y la codificación se aprendieron solo con Train y dentro de cada pliegue de la búsqueda, mediante el preprocesador de las actividades anteriores. Esto evita aprender las transformaciones numéricas a partir de Test (scikit-learn developers, s.f.-a).

Se evaluaron un clasificador trivial, un árbol libre, uno podado, uno balanceado y el bosque de clase. Para optimizar el bosque se probaron ocho combinaciones: profundidad 4 o 6, mínimo por hoja 5 o 10 y pesos balanced o sin pesos. Se mantuvieron 100 árboles y semilla 42. La búsqueda se hizo sobre Train; Test no se usó para escoger parámetros.

Accuracy de entrenamiento y prueba

Modelo	Train (%)	Test (%)	Brecha (pp)
Árbol libre	75,76	71,27	4,49
Árbol podado	73,32	73,32	0,00

Resultados sobre Test

Modelo	Acc.	F1	Recall	AUC
Siempre estándar	0.733	0.000	0.000	0.500
Árbol libre	0.713	0.201	0.135	0.649
Árbol podado	0.733	0.000	0.000	0.669
Árbol balanceado	0.600	0.498	0.744	0.678
Bosque de clase	0.562	0.495	0.804	0.689
Bosque optimizado	0.585	0.500	0.777	0.692

IV ANÁLISIS DE RESULTADOS

El árbol libre obtuvo 75,76 % en Train y 71,27 % en Test: una brecha de 4.49 puntos porcentuales. En el podado fue prácticamente cero, pero no detectó ningún élite. El accuracy 73,32 % coincide con predecir siempre estándar. Por eso la poda no demuestra por sí sola un buen ajuste para el objetivo de scouting.

La búsqueda eligió profundidad 6, mínimo de 10 jugadores por hoja y pesos balanced. El F1 medio de validación fue 0.5059. En Test, el F1 del bosque subió de 0.4948 a 0.5001; su AUC pasó de 0.6888 a 0.6921. El árbol balanceado obtuvo AUC 0.6779. La mejora del bosque es moderada y el AUC final de 0.6921 aún muestra capacidad discriminativa limitada.

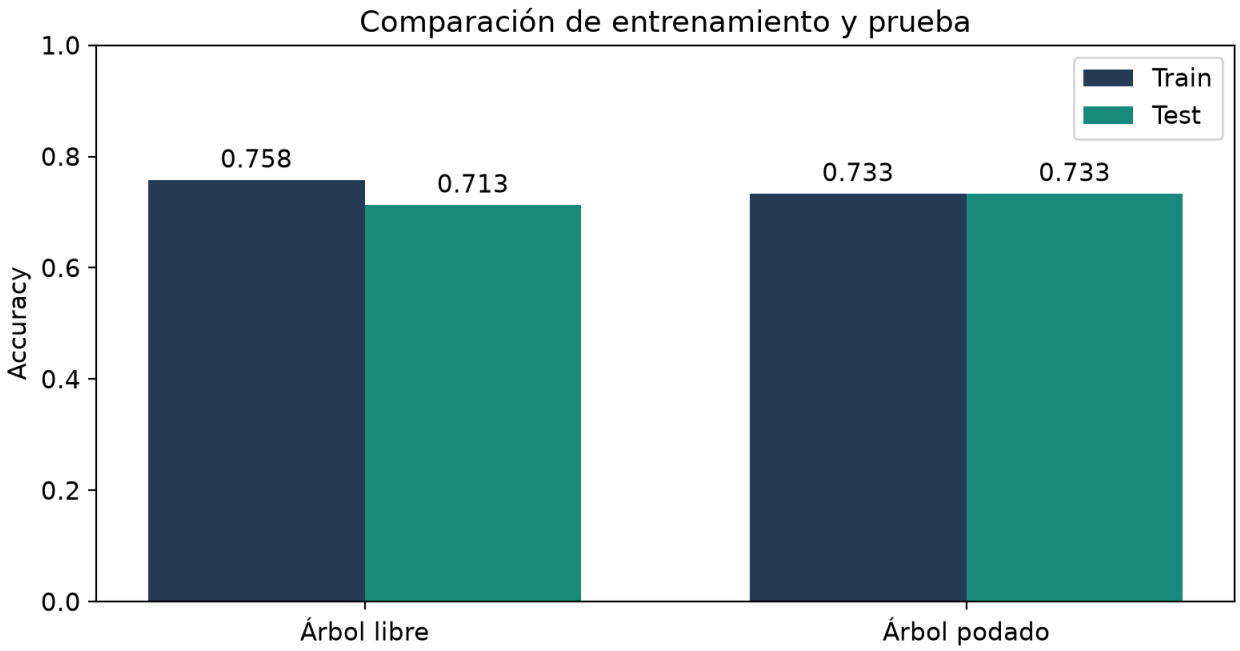
El bosque optimizado encontró 1719 de 2211 élites, con recall 77,75 % y precisión 36,86 %. Frente al árbol libre, los falsos negativos bajaron de 1912 a 492, mientras los falsos positivos subieron de 469 a 2945. Recupera candidatos que el árbol perdía, a cambio de revisar más perfiles que no pertenecen a la clase élite.

Modelo	TN	FP	FN	TP
Árbol libre	5608	469	1912	299
Bosque optimizado	3132	2945	492	1719

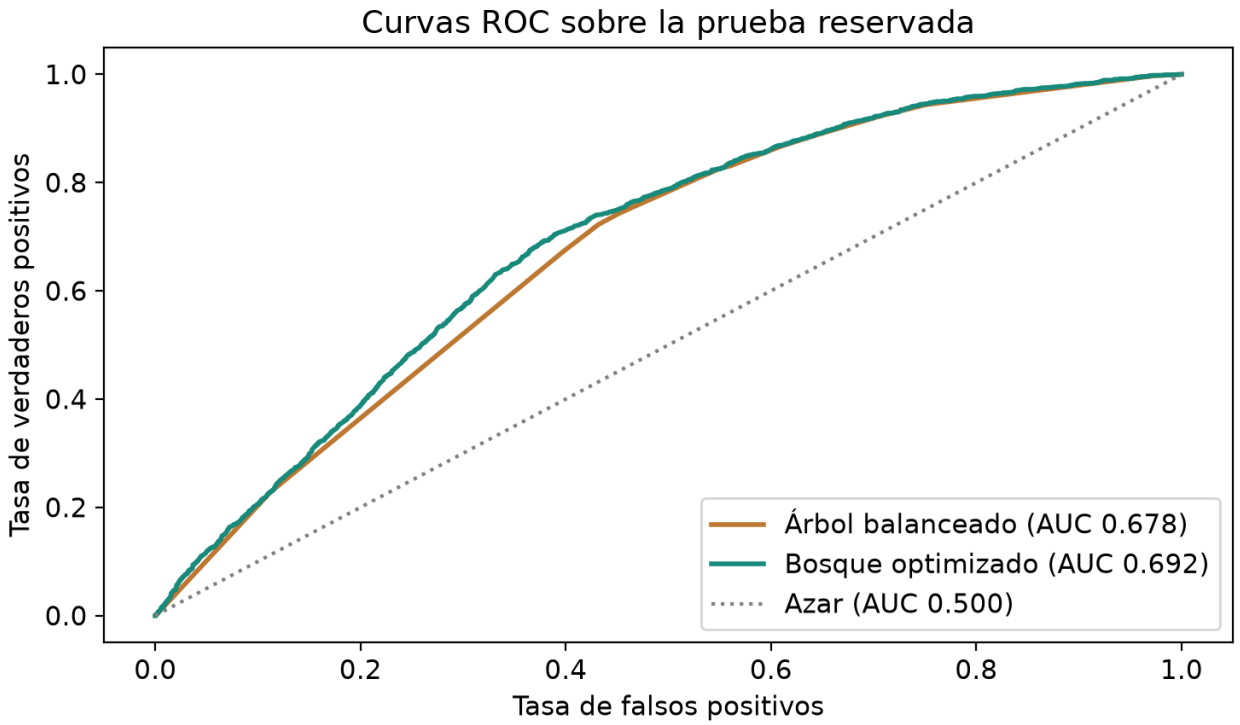
La edad concentró el 81,50 % de la importancia Gini; la posición de portero, 8,78 %, y la estatura, 7,65 %. Una interpretación posible es que la edad distinga etapas de la trayectoria comercial de los jugadores registrados. El gráfico no demuestra que cambiar la edad o la estatura cause un cambio de precio. La presencia de jugadores de distintas épocas y la definición de élite también pueden influir en el patrón.

Limitaciones. Solo se consideran edad, estatura y posición, sin rendimiento reciente, minutos, lesiones, contratos ni contexto competitivo. Se utiliza una fotografía de los datos y una partición aleatoria, no una validación temporal. El percentil global se conserva como definición de la cohorte del curso; para estudiar transferibilidad debe fijarse previamente en una muestra de desarrollo y evaluarse en otra cohorte. Las probabilidades no se calibraron.

V GRÁFICAS DE EVALUACIÓN

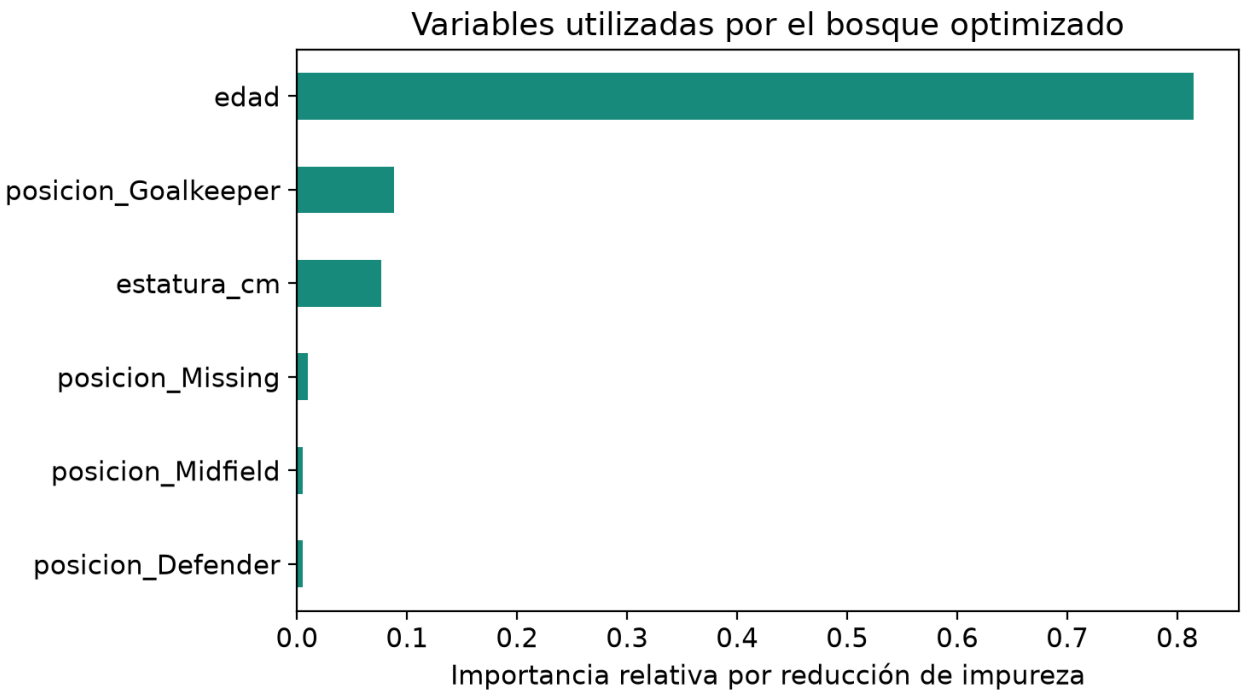


Comparación real de Train y Test. El árbol libre muestra una brecha cercana a 4,49 puntos. El podado reduce esa diferencia, aunque su recall de élite sigue en cero.



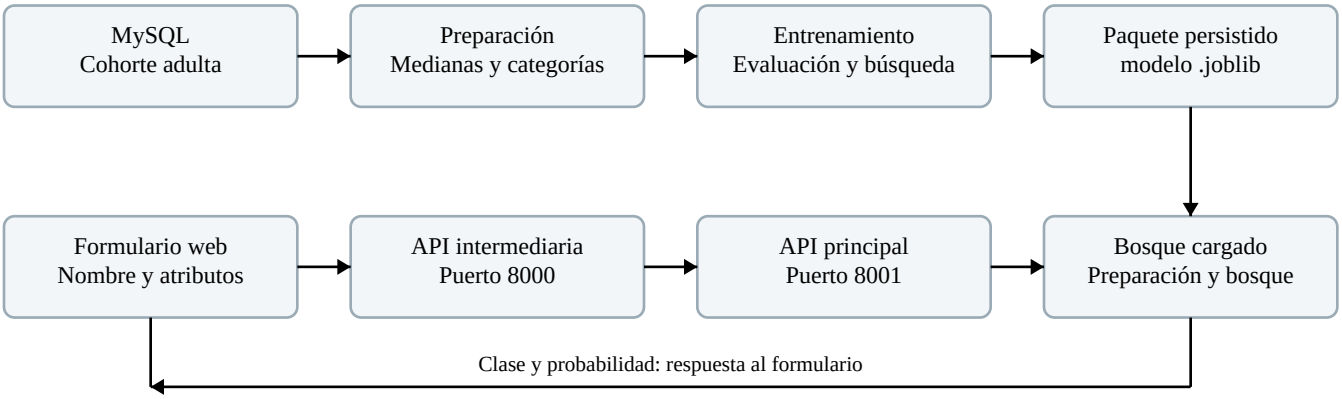
Curvas sobre la misma prueba reservada. El bosque optimizado alcanza AUC 0,6921 frente a 0,6779 del árbol balanceado. La diagonal representa el ordenamiento aleatorio.

VI IMPORTANCIA DE VARIABLES Y ARQUITECTURA



Importancia por reducción de impureza del bosque exportado. La edad predomina; las otras variables aportan menos. Estas importancias describen el modelo y no son evidencia de causalidad.

Flujo de entrenamiento y consulta



La parte superior corresponde al entrenamiento. En cada consulta, la API intermediaria reenvía los datos a main_api.py y devuelve su respuesta al formulario. La API principal carga el bosque una vez y aplica las medianas y categorías guardadas para calcular la clase y su probabilidad.

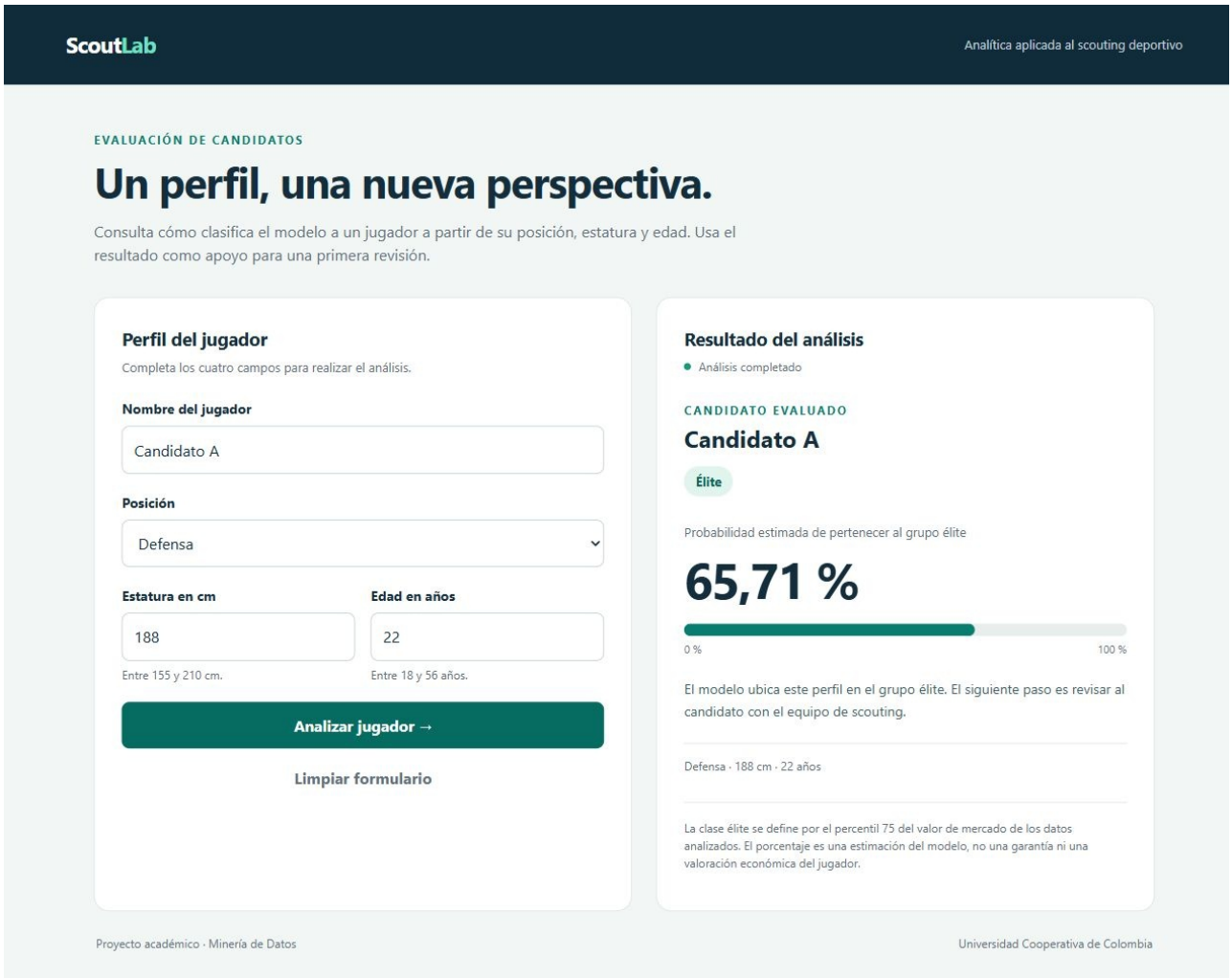
El paquete conserva las categorías y medianas aprendidas. Al consultar un jugador se fijan todas las categorías antes de aplicar get_dummies y drop_first, para mantener los indicadores y el orden de columnas del entrenamiento. Se comprobó que las cuatro posiciones producen vectores distintos y que la inferencia individual coincide con la realizada en lote.

El navegador consulta la intermediaria en el puerto 8000 y esta se comunica con la principal en 127.0.0.1:8001. La principal valida nombre y posición, edad entera entre 18 y 56 años y estatura entre 155 y 210 cm. El nombre identifica al candidato. Las predicciones utilizan el modelo guardado y no requieren consultar MySQL.

El navegador puede inspeccionar la ruta de la intermediaria. La configuración de ambas APIs limita su acceso al equipo local. El proyecto no incorpora autenticación para publicarlo en internet.

VII APLICATIVO Y API EN EJECUCIÓN

Prueba A: candidato de 22 años, defensa y 188 cm. La consulta real devolvió Élite y una probabilidad estimada de 65,71 %. Es un perfil de prueba, no un jugador identificado de la base.



Formulario conectado al modelo serializado. El resultado se obtiene con fetch y se muestra con una barra de progreso. El diseño también se comprobó en el ancho móvil del navegador.

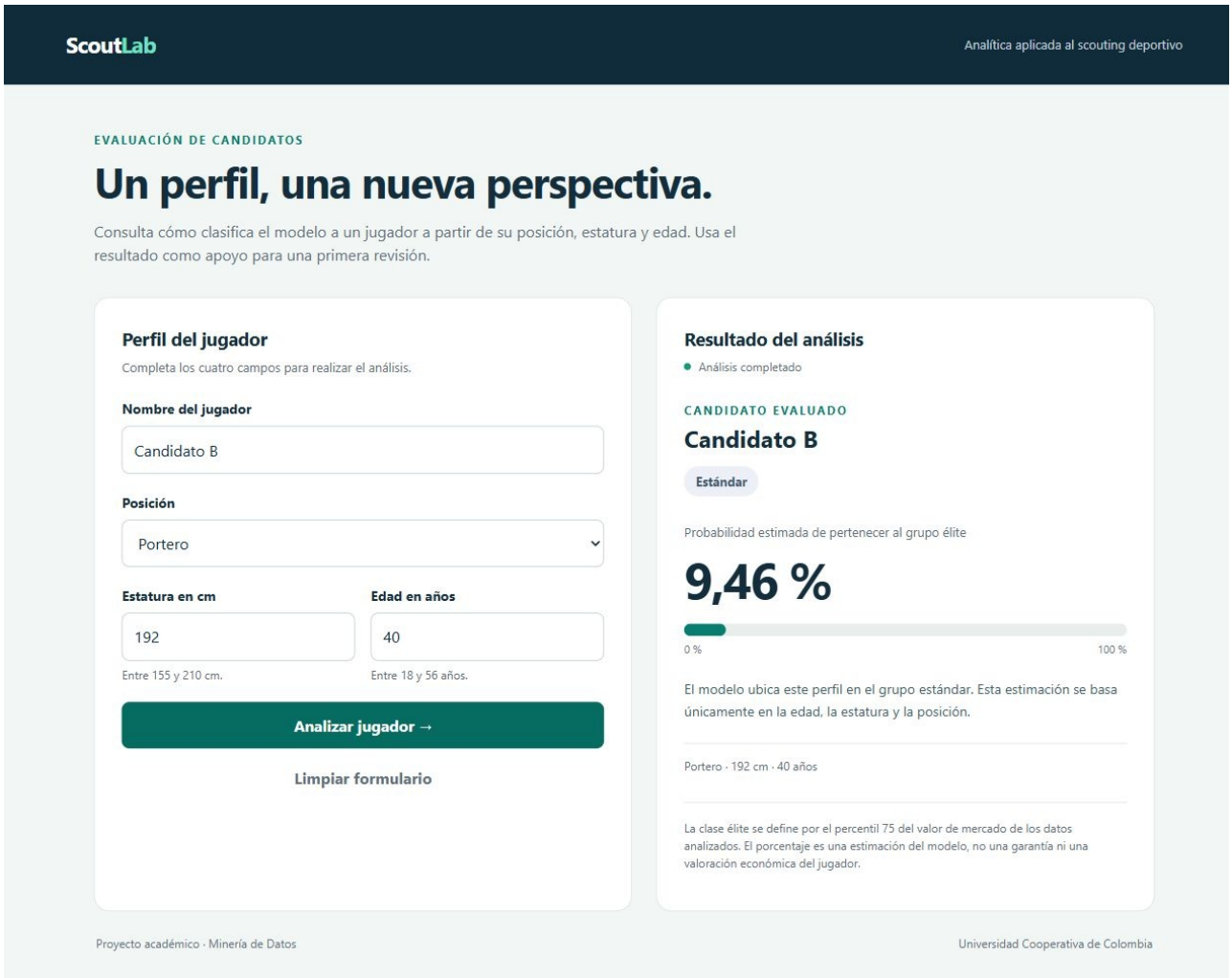
Dar formato al texto ☐


```
{"estado":"listo","modelo":"Random Forest","arboles":100}
```


Respuesta real de GET /health durante la ejecución: servicio listo, Random Forest y 100 árboles. Las capturas del proyecto documentan la aplicación funcionando.

VIII PRUEBAS FUNCIONALES

Prueba B: candidato de 40 años, portero y 192 cm. La respuesta fue Estándar, con 9,46 % de probabilidad estimada de élite. Ambos ejemplos se procesaron con el mismo paquete evaluado.



Segundo perfil de prueba. El porcentaje es una estimación del clasificador y no garantiza un valor de mercado ni una decisión de contratación.

Se verificaron la carga del modelo, la conservación de medianas y categorías, la codificación de las posiciones y la igualdad de la inferencia individual y en lote. Los ejemplos mantuvieron sus probabilidades después de integrar la preparación en los scripts de exportación y consulta.

Las consultas aceptan edades enteras de 18 a 56 años y estaturas de 155 a 210 cm. Son los extremos de la base revisada el 8 de noviembre de 2026, que incluye jugadores históricos. Para fijar el mínimo de estatura se excluyeron once valores atípicos de 17, 18 y 19 cm. Estos límites corresponden a las entradas del aplicativo.

La intermediaria se probó enviando consultas a la principal. Los resultados coincidieron con la consulta directa. Se verificaron respuestas de error cuando la principal está apagada o tarda demasiado en responder.

Se comprobaron los cuatro extremos del rango y se rechazaron 100 años y 300 cm tanto en el formulario como en la API. Las entradas fuera de rango reciben HTTP 422 y los perfiles válidos conservan sus probabilidades.

IX CONCLUSIONES Y RECOMENDACIONES

El proyecto completa el flujo exigido por la actividad: consulta relacional, preparación, diagnóstico, optimización del bosque, exportación y uso desde un aplicativo. Los resultados muestran que el accuracy aislado era insuficiente: el modelo trivial obtuvo 73,32 % sin detectar élites. El bosque optimizado aumentó la recuperación de esa clase, aunque produjo muchas falsas alarmas.

Para un proceso de scouting, puede utilizarse como una primera lista de perfiles a revisar si el costo de revisar candidatos adicionales es menor que el de perder oportunidades. La precisión de 36,86 % y el AUC de 0,6921 no justifican decisiones automáticas de contratación. Deben combinarse con observación deportiva y criterios definidos por el equipo.

El peso de la edad y las pocas variables disponibles exigen cautela al interpretar el resultado como potencial deportivo. La etiqueta solo refleja el segmento comercial definido. Se recomienda ampliar información relevante y obtener una prueba temporal o externa antes de prometer utilidad fuera de la cohorte analizada.

En despliegue se deben monitorear cambios en las distribuciones de entrada, o data drift, y en su relación con la etiqueta, o concept drift. También conviene estudiar calibración de probabilidades y revisar los costos de falsos positivos y negativos. Estas son recomendaciones posteriores; no se presentan como experimentos ya realizados (Mejia Villalba, 2026b).

REFERENCIAS

Breiman, L. (2001). Random forests. *Machine Learning, 45*, 5-32. https://doi.org/10.1023/A:1010933404324

FastAPI. (s.f.). *Body - Fields.* Recuperado el 31 de octubre de 2026, de https://fastapi.tiangolo.com/tutorial/body-fields/

Mejia Villalba, R. E. (2026a). *Actividad final* [Guía de curso]. Universidad Cooperativa de Colombia.

Mejia Villalba, R. E. (2026b). *Guía docente y estructura de la clase Semana 9* [Material de clase]. Universidad Cooperativa de Colombia.

scikit-learn developers. (s.f.-a). *Common pitfalls and recommended practices.* Recuperado el 31 de octubre de 2026, de https://scikit-learn.org/stable/common_pitfalls.html

scikit-learn developers. (s.f.-b). *Metrics and scoring: Quantifying the quality of predictions.* Recuperado el 31 de octubre de 2026, de https://scikit-learn.org/stable/modules/model_evaluation.html

scikit-learn developers. (s.f.-c). *Model persistence.* Recuperado el 31 de octubre de 2026, de https://scikit-learn.org/stable/model_persistence.html

scikit-learn developers. (s.f.-d). *RandomForestClassifier*. Recuperado el 31 de octubre de 2026, de https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

APÉNDICE. IMPLEMENTACIÓN DE SCOUTLAB

El proyecto tiene dos flujos. En el primero se preparan los datos y se guarda el bosque. En el segundo el navegador envía un perfil y recibe la predicción del bosque guardado.

A. Extracción y preparación de jugadores

src/database/connection.py toma la configuración de MySQL del entorno. **exportar_modelo.py** consulta la tabla *players*, calcula los años cumplidos con la fecha de nacimiento y selecciona la cohorte adulta. La edad, la estatura y la posición son las entradas del modelo. El valor de mercado se usa para construir la etiqueta *es_elite* con el percentil 75 de la cohorte.

Separamos 80 % para entrenamiento y 20 % para prueba, manteniendo la proporción de las clases. El *Pipeline* aprende las medianas y las categorías con la parte de entrenamiento de cada pliegue. Completa la estatura faltante con la mediana por posición y usa la mediana global como respaldo. La posición se convierte en indicadores binarios.

B. Comparación, ajuste y exportación

Retomamos la comparación entre el árbol libre y el podado de las prácticas. También probamos un clasificador que siempre predice Estándar para mostrar por qué el *accuracy* aislado puede ocultar que se pierden todos los élite. Después evaluamos el árbol balanceado y el bosque de clase. *GridSearchCV* comparó ocho configuraciones con cinco pliegues estratificados y eligió la de mayor F1 medio.

El ganador tuvo 100 árboles, profundidad máxima de 6, un mínimo de 10 muestras por hoja y pesos balanceados. Se midió con los datos de prueba reservados y se guardó en **models/modelo_scouting_rf.joblib**, junto con las medianas, las categorías y el orden de columnas. El exportador también genera las métricas, la validación cruzada y las gráficas en **resultados/**.

C. Cómo llega una predicción al navegador

index.html recoge nombre, posición, estatura y edad, y envía JSON con *fetch* a **api_intermediaria.py** en el puerto 8000. La intermediaria reenvía la solicitud a **main_api.py** en el puerto 8001. La API principal valida la entrada, recrea las columnas aprendidas, calcula la clase y devuelve las probabilidades. La página actualiza la barra y muestra el nombre para identificar el resultado.

La consulta usa el archivo del modelo que se carga al iniciar. Para volver a entrenar se necesita MySQL y una configuración de conexión, que puede cargarse desde un archivo **.env** local.

D. Demostración con el modelo entregado

Desde la carpeta del proyecto y con las dependencias instaladas, se abre una terminal por comando.

```
python -m uvicorn main_api:app --port 8001
python -m uvicorn api_intermediaria:app --port 8000
```

Al abrir http://127.0.0.1:8000, un defensa de 22 años y 188 cm obtiene 65,71 % de probabilidad estimada de élite. Un portero de 40 años y 192 cm obtiene 9,46 %. Estos resultados sirven para decidir qué perfiles revisar primero.